

COSMIC SHORE · ENGINEERING

Building unbreakable multiplayer

How Cosmic Shore wires Unity Netcode + Unity Gaming Services — the architecture, the four decisions it hangs on, and the bugs that taught us the most.

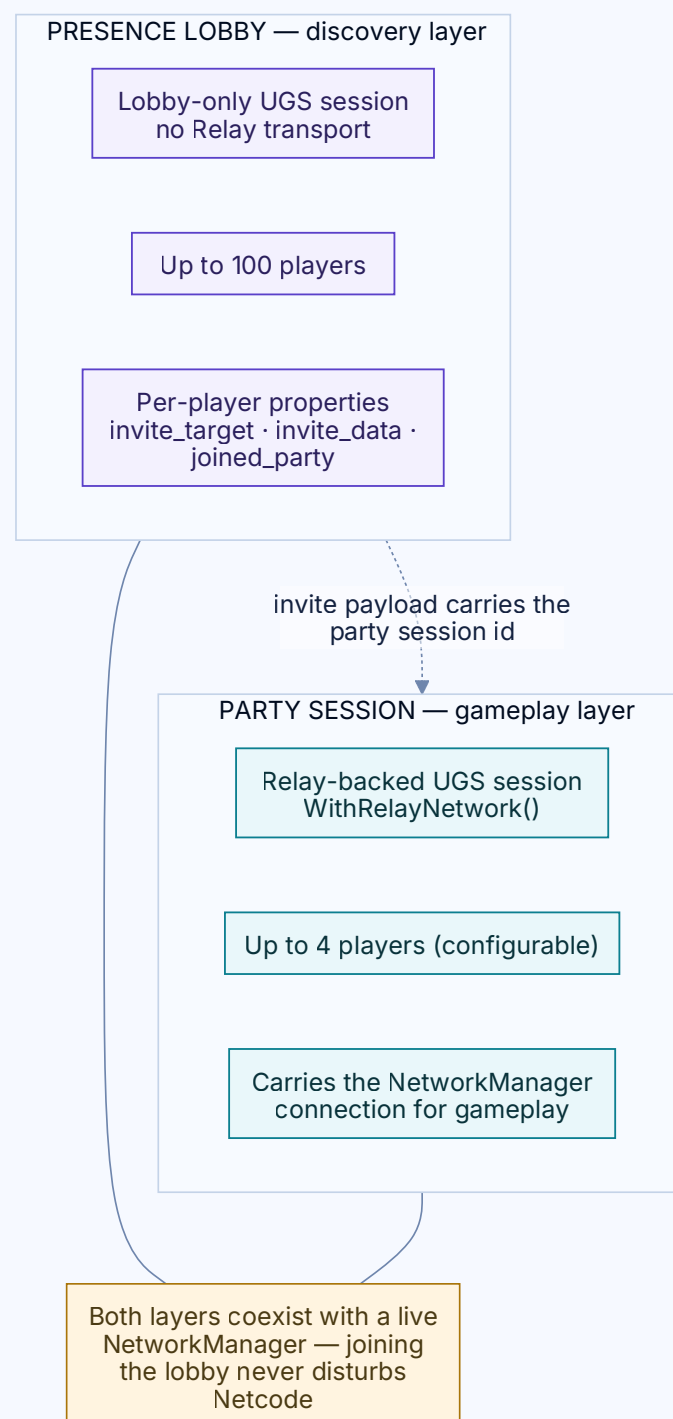
Swipe →



Split “discoverable” from “connected”

Two sessions per player: a cheap, global **Presence Lobby** for discovery and invites, and a Relay-backed **Party Session** for the people actually flying together.

Discovery vs. gameplay

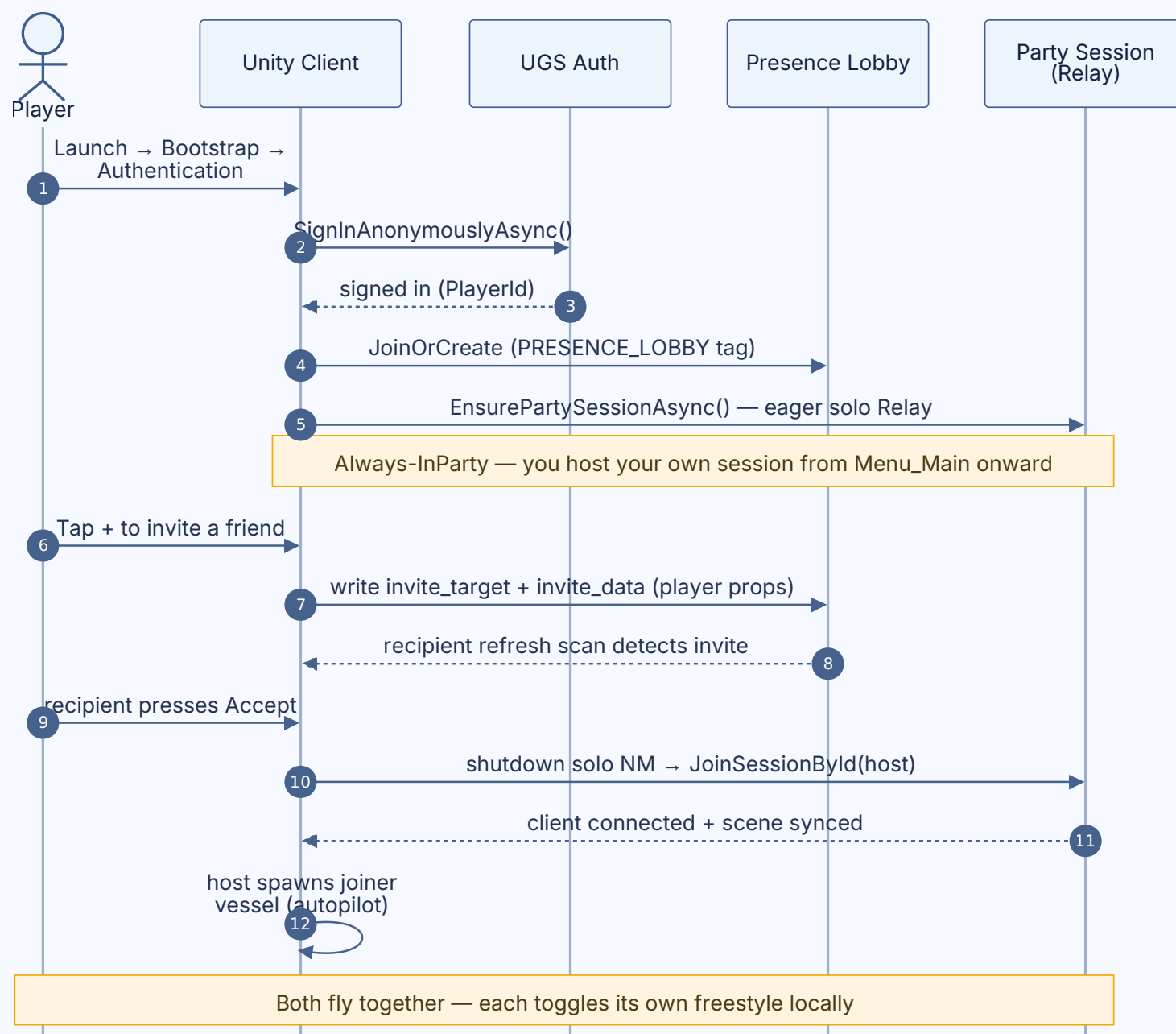


Lobby-only discovery costs no Relay and never disturbs Netcode. The invite payload is the bridge between the two layers.

Everyone hosts a party from the start

EAGER "Always-InParty": create your Relay session the moment you reach the menu. Now an invite is a simple **JOIN** — not a fragile create-then-handoff. An entire class of race-condition bugs simply disappears.

An invite becomes a join



Because a real session already exists, accepting is just "leave mine, join yours."

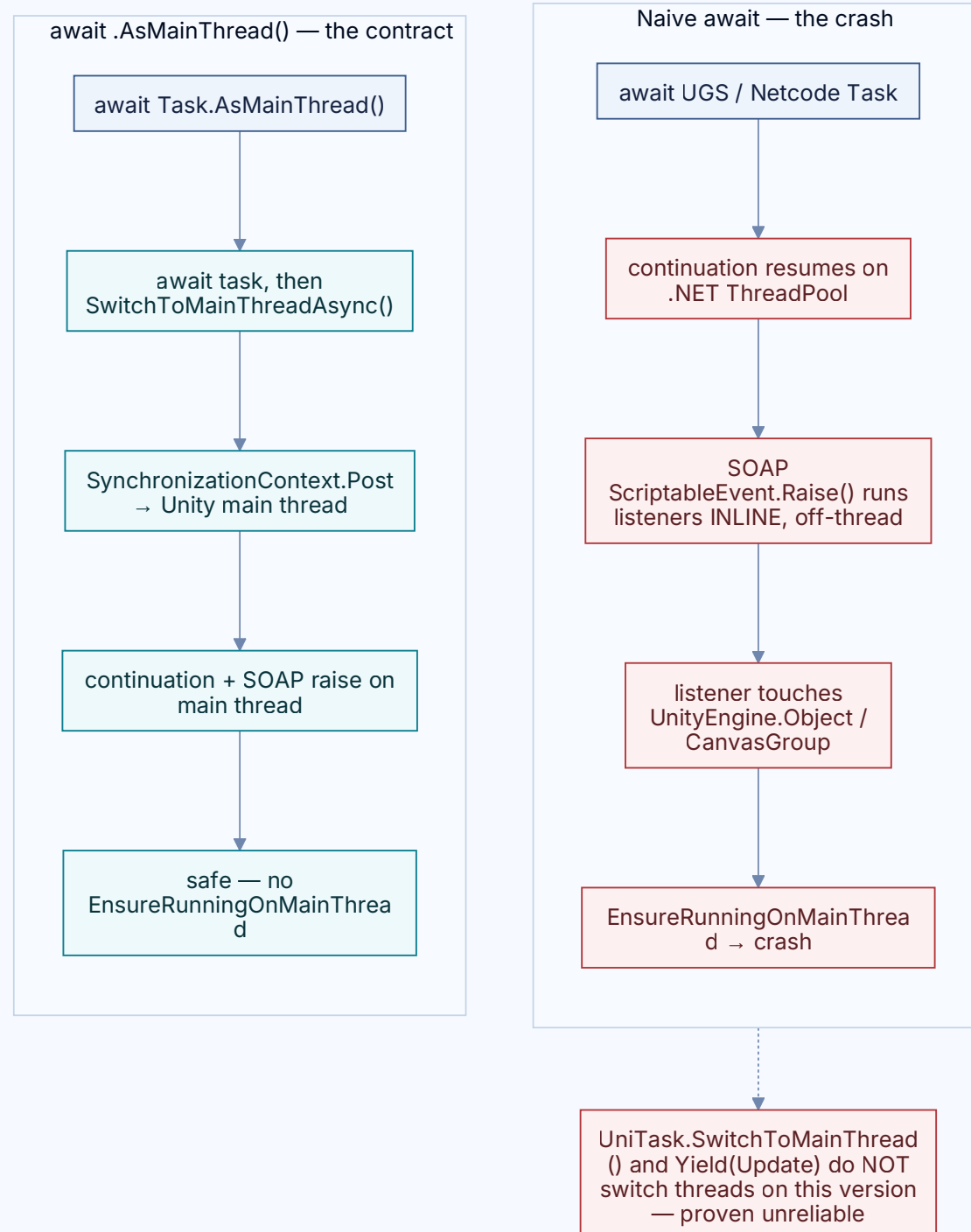
Your await came back on the wrong thread

UGS callbacks resume on the **thread pool**. Touch any Unity object there — even `obj == null` — and it crashes.

Worse: the obvious fix, `UniTask.SwitchToMainThread()`, was a **no-op** on our version.

```
// back on Unity's main thread:  
var s = await Multiplayer.Instance  
    .CreateSessionAsync(opts)  
    .AsMainThread();
```

.AsMainThread() at every cloud await



A boundary helper built on Unity's own SynchronizationContext — plus a canary that screams if anyone forgets it.

A singleton pinned to null — forever

Caching `MultilayerService.Instance` in a constructor that runs **before** UGS initialises pins **null** permanently. Resolve it lazily instead.

```
//      ctor runs before UGS init → null forever
//      resolve at use time:
private IMultilayerService _svc ⇒
    MultilayerService.Instance;
```


When two truths disagree

A player who left kept **flickering back** on the host every few seconds. A presence-lobby “hint” was overriding the authoritative session. **Fix:** pick one source of truth — the session — and make the hint defer to it.

Three disciplines

- ◆ **Single-writer SOAP** — one owner per piece of shared state; everyone else reads.
- ◆ **A validated state machine** — 7 explicit party states, not a drift-prone scatter of booleans.
- ◆ **Classify every failure** — each catch maps to a named recovery: benign · rate-limit · gone · transient.

8 “unbreakable” criteria, checked every commit

- ◆ No fatal failure · no stuck UI · no silent state divergence
- ◆ Every transition reversible · idempotent retries (double-tap is safe)
- ◆ 3-player & 4-player concurrent-invite tests green in Multiplayer Play Mode

5 reusable lessons

- ◆ Delete a fragile transition by making its precondition **always true**.
- ◆ Verify your async library's thread-switch — **don't trust it**.
- ◆ Never cache a singleton before its init can complete.
- ◆ One authoritative source; everything else **defers**.
- ◆ Order object vs. scene lifecycles explicitly, or get orphans.

What we'd improve next

- ◆ **Host-loss resilience** — today the host is a player; if they drop, the party ends.
- ◆ **Prove 3–4-player parties** — close the second-joiner edge case.
- ◆ **Push over polling** — event-driven invites to cut latency + rate-limit churn.
- ◆ **Production telemetry** — measure party success and join latency in shipped builds.

The party game for pilots

Built by **Froglet Inc.** on Unity 6, Netcode for GameObjects & Unity Gaming Services.

Full 69-page technical dossier available on request